ĐẠI HỌC TRÀ VINH

TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ

ĐỒ ÁN HỌC PHẦN: CHUYÊN ĐỀ ASP.NET

ĐỀ TÀI:

HỆ THỐNG TUYỂN DỤNG VIỆC LÀM TRỰC TUYẾN

Giáo viên hướng dẫn:
Giảng viên: TS.
Đoàn Phước Miền

Sinh viên thực hiện:
Họ tên: Kiều Đức Anh
MSSV: 170124533
Lớp: DK24TTC5

Vĩnh Long, tháng 04 năm 2026

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Vĩnh Long, ngày ..... tháng ..... năm .....

Giáo viên hướng dẫn

(Ký tên và ghi rõ họ tên)

NHẬN XÉT CỦA THÀNH VIÊN HỘI ĐỒNG

.....

.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....
.....

....., ngày ..... tháng ..... năm .....

Thành viên hội đồng

(Ký tên và ghi rõ họ tên)

LỜI CẢM ƠN

Trong suốt quá trình thực hiện đồ án với đề tài "Hệ thống điều phối nhân sự JobFinder trên nền tảng ASP.NET Core & Localized Database", nhóm thực hiện đã nhận được rất nhiều sự hỗ trợ quý báu từ phía giảng viên, gia đình và bạn bè. Đây là nền tảng quan trọng giúp đồ án không chỉ được hoàn thiện về mặt kỹ thuật mà còn được định hình rõ ràng về tư duy nghiên cứu, phương pháp tiếp cận và định hướng triển khai thực tiễn.

Trước hết, chúng em xin bày tỏ lòng biết ơn sâu sắc tới ThS. Đoàn Phước Miền, giảng viên hướng dẫn thuộc Khoa Công nghệ thông tin. Trong suốt quá trình triển khai đề tài, thầy đã dành nhiều thời gian góp ý chuyên môn, định hướng cách tiếp cận bài toán dưới góc độ khoa học và kỹ thuật, đồng thời giúp chúng em nhìn nhận rõ hơn yêu cầu về tính đúng đắn, tính thực dụng và tính khả thi của một hệ thống phần mềm phục vụ doanh nghiệp. Những nhận xét của thầy không chỉ tập trung vào bề nổi của giao diện hay tính năng, mà còn nhấn mạnh đến các vấn đề cốt lõi như kiến trúc hệ thống, lựa chọn công nghệ, quản trị dữ liệu và bảo đảm khả năng mở rộng. Đây là những bài học rất quan trọng đối với chúng em trong quá trình học tập và nghiên cứu về Công nghệ phần mềm.

Chúng em cũng xin chân thành cảm ơn tập thể giảng viên Khoa Công nghệ thông tin đã truyền đạt nền tảng kiến thức vững chắc về lập trình, cơ sở dữ liệu, phân tích thiết kế hệ thống, mạng máy tính, kiểm thử phần mềm và quản trị dự án. Những học phần tưởng như tách biệt đã được kết nối lại một cách rõ nét trong quá trình xây dựng JobFinder, khi nhóm phải đồng thời giải quyết các vấn đề từ mô hình dữ liệu, logic nghiệp vụ, xác thực người dùng cho đến thiết kế trải nghiệm giao diện và cấu hình triển khai.

Bên cạnh đó, chúng em xin gửi lời cảm ơn đến gia đình, những người luôn là điểm tựa tinh thần lớn nhất trong suốt chặng đường học tập. Sự động viên, cảm thông và hỗ trợ của gia đình đã giúp chúng em duy trì sự tập trung, bình tĩnh vượt qua các giai

đoạn áp lực của việc hoàn thiện một đồ án có tính tích hợp cao giữa nghiên cứu và lập trình thực tế.

Chúng em cũng không quên cảm ơn bạn bè, đồng môn và những người đã cùng trao đổi, phản biện và chia sẻ kinh nghiệm trong quá trình thực hiện đề tài. Nhiều góp ý về trải nghiệm người dùng, độ hợp lý của luồng đăng nhập, tính trực quan của giao diện tuyển dụng, cũng như các tình huống kiểm thử thực tế đã giúp nhóm hoàn thiện sản phẩm theo hướng gần hơn với nhu cầu vận hành thật.

Mặc dù nhóm đã cố gắng hoàn thiện đồ án với tinh thần nghiêm túc và cầu thị, song do giới hạn về thời gian, nguồn lực và phạm vi triển khai, báo cáo khó tránh khỏi những thiếu sót nhất định. Chúng em rất mong tiếp tục nhận được các ý kiến đóng góp từ thầy cô để có thể hoàn thiện hơn cả về sản phẩm phần mềm lẫn phương pháp nghiên cứu trong tương lai.

Xin trân trọng cảm ơn.

Mục lục

LỜI CẢM ƠN 4

TÓM TẮT 7

CHƯƠNG 1. TỔNG QUAN VỀ HỆ SINH THÁI TUYỂN DỤNG SỐ 9

1.1. Sự chuyển dịch của E-Recruitment 9

1.2. Thực trạng và thách thức của các portal tuyển dụng hiện nay 10

1.3. Nhu cầu cấp thiết đối với Private Recruitment Portal 12

1.4. Cơ sở lựa chọn công nghệ 13

CHƯƠNG 2. PHÂN TÍCH HỆ THỐNG VÀ MÔ HÌNH HÓA 16

2.1. Phân tích chức năng 16

2.2. Phân tích phi chức năng 18

2.3. Thiết kế cơ sở dữ liệu 20

2.4. Mô tả luồng xử lý và sơ đồ logic 23

CHƯƠNG 3. TRIỂN KHAI KỸ THUẬT VÀ THỰC NGHIỆM 26

3.1. Kiến trúc frontend 26

3.2. Logic backend 29

3.3. Đánh giá thực nghiệm 31

3.4. Giao diện người dùng và định hướng thiết kế 34

KẾT LUẬN 36

Hướng phát triển 37

TÀI LIỆU THAM KHẢO 39

TÓM TẮT

Đồ án "Hệ thống điều phối nhân sự JobFinder trên nền tảng ASP.NET Core & Localized Database" được thực hiện nhằm xây dựng một cổng tuyển dụng vận hành độc lập, tối ưu cho bối cảnh triển khai nhẹ, chi phí thấp và tốc độ phản hồi cao. Điểm nhấn kỹ thuật của hệ thống là hướng tiếp cận Localized Data Processing, trong đó dữ liệu nghiệp vụ được lưu trữ và xử lý cục bộ thông qua SQLite thay vì phụ thuộc hoàn toàn vào các hệ quản trị cơ sở dữ liệu tập trung có chi phí vận hành cao như SQL Server, Oracle hay các dịch vụ cloud database chuyên dụng. Cách tiếp cận này đặc biệt phù hợp với các doanh nghiệp vừa và nhỏ, các đơn vị nội bộ cần môi trường demo, hệ thống tuyển dụng dùng trong mạng riêng hoặc các nền tảng cần thời gian khởi tạo hạ tầng ngắn.

Về mặt công nghệ, hệ thống được xây dựng trên ASP.NET Core MVC kết hợp Entity Framework Core. ASP.NET Core cung cấp môi trường thực thi đa nền tảng, hiệu suất tốt, tích hợp sẵn Dependency Injection, pipeline middleware linh hoạt và cơ chế xác thực theo claims. Entity Framework Core được dùng để ánh xạ đối tượng sang dữ liệu quan hệ, giảm khối lượng mã truy cập dữ liệu thủ công, đồng thời cho phép hiện thực hóa ràng buộc dữ liệu ngay tại tầng mô hình thông qua Data Annotations. SQLite đóng vai trò là bộ máy lưu trữ zero-configuration, không yêu cầu cài đặt server riêng, cho phép hệ thống khởi chạy với một tệp `.db` duy nhất và rất phù hợp cho mô hình triển khai container hóa.

Hệ thống JobFinder hiện thực các chức năng cốt lõi gồm: trang chủ hiển thị danh sách việc làm và khối tìm kiếm, trang đăng nhập sử dụng cookie authentication, cơ chế đăng xuất, xem chi tiết việc làm, khởi tạo dữ liệu mẫu tiếng Việt và luồng tái nạp dữ liệu cho tài khoản quản trị. Ở lớp giao diện, đồ án áp dụng phong cách thiết kế trực quan theo tông Professional Blue nhằm gia tăng cảm nhận tin cậy của một nền tảng tuyển dụng, đồng thời sử dụng hiệu ứng Glassmorphism cho trang đăng nhập để nâng cao tính hiện đại. Ở lớp backend, hệ thống được tổ chức theo mô hình MVC giúp phân tách rõ trách nhiệm giữa trình bày, điều phối luồng và dữ liệu.

Giá trị mới của đồ án không nằm ở việc xây dựng một portal tuyển dụng thông thường, mà ở việc chứng minh rằng với Localized Data Processing, một hệ thống tuyển dụng vẫn có thể đáp ứng yêu cầu quản trị người dùng, tra cứu việc làm và trình bày dữ liệu hiệu quả mà không cần phụ thuộc vào hạ tầng dữ liệu tập trung phức tạp. Điều này kéo theo ba lợi ích rõ rệt. Thứ nhất là giảm chi phí sở hữu hạ tầng ban đầu, vì không cần duy trì máy chủ cơ sở dữ liệu riêng. Thứ hai là cải thiện độ linh hoạt khi triển khai, do toàn bộ hệ thống có thể đóng gói bằng Docker và phân phối lên môi trường như Render. Thứ ba là rút ngắn độ trễ ở quy mô nhỏ và trung bình, do truy vấn dữ liệu được thực thi cục bộ với số tầng trung gian ít hơn.

Trong báo cáo này, nhóm tập trung phân tích hệ thống theo hướng kỹ thuật sâu, bao gồm: bối cảnh chuyển dịch của hệ sinh thái tuyển dụng số; các yêu cầu chức năng và phi chức năng; thiết kế schema dữ liệu; cơ chế xác thực; chiến lược khởi tạo dữ liệu; mô tả kiến trúc frontend và backend; cũng như đánh giá thực nghiệm đối với các kịch bản tìm kiếm và truy xuất dữ liệu. Báo cáo đồng thời chỉ ra giới hạn của hướng tiếp cận localized database khi quy mô dữ liệu và số lượng truy cập tăng cao, từ đó đề xuất lộ trình nâng cấp sang kiến trúc cloud-native hoặc microservices trong tương lai.

CHƯƠNG 1. TỔNG QUAN VỀ HỆ SINH THÁI TUYỂN DỤNG SỐ

1.1. Sự chuyển dịch của E-Recruitment

Tuyển dụng là một chức năng cốt lõi của quản trị nguồn nhân lực. Trong giai đoạn trước khi số hóa diễn ra mạnh mẽ, hoạt động tuyển dụng phần lớn được thực hiện thông qua các kênh truyền thống như đăng báo giấy, nhận hồ sơ trực tiếp, phỏng vấn tập trung và xử lý thủ công bằng văn bản hoặc bảng tính. Mô hình này tồn tại nhiều hạn chế: chi phí truyền thông cao, thời gian xử lý kéo dài, khả năng tiếp cận ứng viên hẹp và rất khó chuẩn hóa dữ liệu ứng viên hoặc thống kê hiệu quả tuyển dụng. Khi nhu cầu tuyển dụng tăng về tần suất lẫn độ phức tạp, phương pháp này bộc lộ rõ tính kém hiệu quả.

Sự phát triển của internet, điện toán đám mây và các nền tảng web đã thúc đẩy quá trình chuyển dịch từ tuyển dụng thủ công sang E-Recruitment. Trong mô hình mới, doanh nghiệp có thể đăng tải nhu cầu tuyển dụng lên các cổng thông tin trực tuyến, nhận hồ sơ điện tử, lọc ứng viên theo bộ tiêu chí, quản lý lịch phỏng vấn và theo dõi trạng thái tuyển dụng thông qua phần mềm. Không gian dữ liệu tuyển dụng vì vậy được số hóa, tập trung và có khả năng truy vấn. Sự chuyển dịch này không chỉ là sự thay đổi phương tiện truyền thông, mà là sự tái cấu trúc toàn bộ chuỗi vận hành tuyển dụng theo hướng dữ liệu hóa và tự động hóa.

Ở cấp độ nền tảng, E-Recruitment hiện nay phát triển theo ba hướng chính. Thứ nhất là các job portal công cộng quy mô lớn, nơi nhiều doanh nghiệp cùng đăng tuyển và cạnh tranh trong cùng một thị trường ứng viên. Thứ hai là hệ thống ATS nội bộ, tập trung vào quản lý quy trình tuyển dụng của một tổ chức riêng. Thứ ba là các hybrid portal kết hợp giữa website doanh nghiệp, trang landing riêng cho chiến dịch tuyển

dụng và các dịch vụ liên kết dữ liệu. Trong cả ba hướng này, dữ liệu đóng vai trò trung tâm. Chất lượng schema, hiệu quả truy vấn, tốc độ phản hồi và khả năng triển khai linh hoạt trở thành yếu tố quyết định giá trị thực tiễn của hệ thống.

Một đặc điểm đáng chú ý của giai đoạn hiện nay là xu hướng tuyển dụng không còn dừng ở việc đăng tải thông tin vị trí việc làm. Thay vào đó, hệ thống phải đóng vai trò như một cổng tương tác hai chiều: doanh nghiệp công bố nhu cầu, ứng viên tra cứu vị trí, người quản trị phân loại hồ sơ, và toàn bộ quá trình này tạo ra một tập dữ liệu có thể đo lường. Điều đó dẫn đến yêu cầu ngày càng cao đối với phần mềm tuyển dụng: phải phản hồi nhanh, hoạt động ổn định, dễ bảo trì và triển khai được trong nhiều bối cảnh hạ tầng khác nhau.

JobFinder được đặt trong bối cảnh chuyển dịch nói trên nhưng tiếp cận từ một lát cắt khác: thay vì xây dựng một job portal công cộng với kiến trúc dữ liệu nặng và phụ thuộc nhiều dịch vụ tập trung, hệ thống tập trung vào mô hình portal tuyển dụng gọn nhẹ, cài đặt nhanh, triển khai độc lập và tối ưu cho các tổ chức cần hệ thống riêng. Đây là một hướng tiếp cận có tính thực dụng cao trong bối cảnh không phải doanh nghiệp nào cũng có khả năng đầu tư vào hạ tầng dữ liệu lớn ngay từ đầu.

1.2. Thực trạng và thách thức của các portal tuyển dụng hiện nay

Mặc dù thị trường đã có nhiều nền tảng tuyển dụng, nhưng các hệ thống hiện có vẫn tồn tại nhiều vấn đề khi xét từ góc độ doanh nghiệp nhỏ và vừa. Vấn đề đầu tiên là độ nặng của cấu trúc dữ liệu và số lượng thành phần hạ tầng đi kèm. Nhiều nền tảng được xây dựng để phục vụ quy mô cực lớn, nên schema dữ liệu có xu hướng mở rộng theo nhiều chiều: ứng viên, hồ sơ, kỹ năng, lịch sử ứng tuyển, chiến dịch truyền thông, phân quyền nhiều tầng, phân tích dữ liệu, thông báo, tích hợp với CRM hoặc HRM. Cấu trúc dữ liệu này phù hợp với enterprise-scale system nhưng tạo ra gánh

nặng đáng kể về thiết kế, quản trị và chi phí vận hành nếu áp dụng nguyên vẹn cho một portal nhỏ.

Thách thức thứ hai là độ trễ khi truy vấn dữ liệu tập trung. Với các hệ thống sử dụng cơ sở dữ liệu đặt riêng trên server hoặc cloud instance, mỗi truy vấn từ ứng dụng web phải đi qua mạng, lớp kết nối, pool quản lý, truy vấn SQL, serialize kết quả và truyền ngược trở lại. Khi số lượng bảng liên kết lớn hoặc cấu trúc schema phân mảnh, độ trễ càng dễ tăng. Trong các hệ thống tuyển dụng, người dùng rất nhạy cảm với tốc độ hiển thị danh sách công việc và chi tiết vị trí. Một độ trễ nhỏ nhưng lặp lại liên tục có thể làm giảm trải nghiệm và ảnh hưởng trực tiếp đến tỉ lệ tương tác.

Thách thức thứ ba là chi phí duy trì các hệ quản trị cơ sở dữ liệu truyền thống như SQL Server hoặc Oracle. Các nền tảng này mạnh, ổn định và phù hợp với bài toán lớn, nhưng không phải lúc nào cũng tương xứng với nhu cầu thực tế của tổ chức. Chi phí cấp phép, tài nguyên máy chủ, vận hành, sao lưu, bảo trì, cập nhật và quản lý an toàn dữ liệu có thể trở thành gánh nặng. Với doanh nghiệp vừa và nhỏ, đặc biệt là các tổ chức mới số hóa quy trình tuyển dụng, việc triển khai một cổng việc làm độc lập đôi khi bị trì hoãn không phải vì khó viết phần mềm, mà vì chi phí hạ tầng đi kèm quá lớn so với kỳ vọng ban đầu.

Một vấn đề khác là tính phụ thuộc vào môi trường triển khai. Nhiều hệ thống tuyển dụng hiện nay được thiết kế theo hướng giả định rằng doanh nghiệp có sẵn DBA, hạ tầng cloud, backup server và quy trình vận hành ứng dụng bài bản. Trong khi đó, thực tế ở nhiều đơn vị là nhu cầu chỉ dừng lại ở việc có một cổng tuyển dụng riêng để hiển thị vị trí đang mở, cho phép người dùng tra cứu, và cho phép bộ phận quản trị cập nhật hoặc làm mới dữ liệu mẫu. Ở mức yêu cầu này, một giải pháp "nhẹ nhưng đủ" có giá trị hơn một nền tảng "đầy đủ nhưng quá nặng".

Về mặt kỹ thuật, nhiều portal còn đối mặt với khó khăn trong việc tối ưu hóa luồng dữ liệu demo hoặc staging. Môi trường trình diễn thường cần khả năng reset nhanh về trạng thái chuẩn để phục vụ giảng dạy, thuyết trình, kiểm thử hoặc onboarding. Nếu sử dụng database server truyền thống, quy trình reset thường kéo theo script dọn dữ liệu, quyền truy cập, backup snapshot và nhiều thao tác vận hành. Điều đó làm tăng độ phức tạp của việc quản lý hệ thống.

Chính từ các thách thức trên, nhóm đặt ra câu hỏi nghiên cứu: liệu có thể xây dựng một hệ thống tuyển dụng riêng biệt, có tốc độ phản hồi tốt, dễ triển khai, sử dụng hạ tầng dữ liệu cực gọn nhưng vẫn đáp ứng đủ các chức năng thiết yếu của một portal tuyển dụng hiện đại hay không. Đồ án JobFinder được phát triển để trả lời câu hỏi này bằng cách kết hợp ASP.NET Core MVC với SQLite theo triết lý localized database.

1.3. Nhu cầu cấp thiết đối với Private Recruitment Portal

Trong nhiều doanh nghiệp, đặc biệt là doanh nghiệp vừa và nhỏ hoặc tổ chức có nhu cầu tuyển dụng theo đợt, việc phụ thuộc hoàn toàn vào các cổng tuyển dụng công cộng tạo ra một số bất lợi. Trước hết là mất kiểm soát về trải nghiệm thương hiệu. Khi doanh nghiệp đăng tuyển trên portal công cộng, giao diện, cấu trúc trình bày, cách hiển thị logo và nội dung thường tuân theo chuẩn chung của nền tảng. Điều này khiến employer branding không được thể hiện trọn vẹn. Một private portal giúp doanh nghiệp kiểm soát hoàn toàn cách hiển thị nội dung tuyển dụng, thiết kế giao diện, cấu trúc điều hướng và cách kết nối với hệ sinh thái truyền thông riêng.

Thứ hai là quyền kiểm soát dữ liệu. Trên các nền tảng công cộng, dữ liệu tuyển dụng và tương tác của ứng viên phần lớn nằm trên hạ tầng của bên thứ ba. Doanh nghiệp không dễ dàng tùy chỉnh schema, quy trình xử lý hoặc các chỉ số theo dõi nội bộ. Trong khi đó, một portal riêng cho phép tổ chức định nghĩa cấu trúc dữ liệu đúng với

nhu cầu, triển khai các cơ chế xác thực và quản trị phù hợp, đồng thời giữ dữ liệu trong môi trường mà doanh nghiệp kiểm soát được.

Thứ ba là nhu cầu triển khai độc lập. Trong các bối cảnh như hội chợ việc làm, demo nội bộ, hệ thống tuyển dụng đặt tại chi nhánh, môi trường đào tạo hoặc thử nghiệm chức năng mới, khả năng đóng gói hệ thống thành một dịch vụ đơn lẻ là rất quan trọng. Với localized database, toàn bộ ứng dụng và dữ liệu có thể được phân phối dưới dạng container hoặc gói chạy cục bộ mà không cần thiết lập nhiều dịch vụ nền đi kèm. Điều này rút ngắn đáng kể thời gian triển khai và đơn giản hóa vận hành.

Thứ tư là tốc độ phản hồi. Trong kiến trúc localized database, ứng dụng và dữ liệu có thể nằm rất gần nhau về mặt triển khai, thậm chí cùng trong một container hoặc cùng host. Do đó thời gian truy xuất dữ liệu được rút ngắn, đặc biệt với các truy vấn nhỏ và vừa, như truy vấn danh sách việc làm, chi tiết vị trí và xác thực người dùng. Với những portal có tập dữ liệu giới hạn nhưng cần phản hồi nhanh, đây là lợi thế thực tế.

Cuối cùng là khả năng mở rộng có kiểm soát. Một private portal không nhất thiết phải đạt khả năng chịu tải như các job marketplace công cộng ngay từ đầu. Quan trọng hơn là nó phải có nền tảng kiến trúc rõ ràng để có thể mở rộng khi cần. Việc dùng MVC, EF Core và cấu trúc code tách lớp tạo ra khả năng chuyển đổi dần dần: từ hệ thống cục bộ sang hệ thống có API, từ SQLite sang một RDBMS mạnh hơn, hoặc từ monolith sang kiến trúc dịch vụ khi quy mô dữ liệu và lượng truy cập vượt ngưỡng.

1.4. Cơ sở lựa chọn công nghệ

Việc lựa chọn công nghệ trong đồ án được thực hiện theo tiêu chí: chi phí vận hành thấp, cấu hình đơn giản, hiệu quả thực thi tốt, có thể triển khai cross-platform và dễ mở rộng về sau.

ASP.NET Core được chọn làm nền tảng phát triển backend vì đây là framework web hiện đại của Microsoft, hỗ trợ đa nền tảng, hiệu suất cao, middleware pipeline rõ ràng, DI tích hợp sẵn và mô hình lập trình nhất quán. Dù phiên bản hiện thực trong mã nguồn của đồ án đang là .NET 6, kiến trúc được xây dựng tương thích về tư duy để có thể nâng cấp lên .NET 8 trong tương lai. Khi phân tích ở góc độ công nghệ, .NET 8 có ưu thế rõ về vòng đời hỗ trợ dài hơn, hiệu năng runtime tốt hơn và nhiều cải tiến liên quan tới observability, trimming và container optimization. Tuy nhiên, bản chất kiến trúc MVC, EF Core và cookie authentication mà đồ án sử dụng không phụ thuộc vào một chi tiết phiên bản duy nhất; chúng vẫn phản ánh đúng hướng tiếp cận công nghệ mà đề tài đặt ra.

Mô hình MVC được lựa chọn thay vì viết tất cả logic trong một lớp xử lý lớn vì nó tạo ra sự phân tách trách nhiệm rõ ràng. Controller phụ trách điều hướng request và phối hợp dữ liệu; Model mô tả thực thể nghiệp vụ và ràng buộc; View chịu trách nhiệm trình bày. Trong bài toán tuyển dụng, sự phân tách này rất cần thiết vì giao diện thường thay đổi theo chiến dịch tuyển dụng, trong khi phần truy vấn dữ liệu và xác thực có yêu cầu ổn định hơn. MVC cũng phù hợp cho môi trường giảng dạy và đồ án vì cấu trúc thư mục, luồng request-response và điểm mở rộng dễ quan sát, dễ kiểm thử.

SQLite được chọn làm cơ sở dữ liệu vì nó đáp ứng đúng triết lý zero-configuration và localized storage. Không cần tiến trình server độc lập, không yêu cầu cấu hình phức tạp, không cần mở cổng dịch vụ riêng. Ứng dụng chỉ cần một tệp `.db` để lưu trữ dữ liệu. Điều này làm giảm đáng kể chi phí thiết lập môi trường và rất phù hợp với mô hình container trên Render hoặc môi trường demo nội bộ. Ngoài ra, ở quy mô dữ liệu nhỏ và trung bình, SQLite có hiệu năng rất tốt cho truy vấn đọc, đặc biệt khi schema được thiết kế hợp lý và truy vấn không lồng ghép quá sâu.

Entity Framework Core được dùng như một lớp ORM giúp đồng bộ giữa mô hình đối tượng và dữ liệu quan hệ. Trong đồ án, EF Core không chỉ giảm khối lượng mã truy vấn SQL thủ công mà còn tạo ra một lớp trừu tượng giúp dữ liệu dễ bảo trì. Quan trọng hơn, Data Annotations trên model cho phép đặt ràng buộc ngay tại tầng ứng dụng, từ đó giảm rủi ro dữ liệu không hợp lệ xâm nhập vào hệ thống.

Về bảo mật, cookie authentication dựa trên claims được chọn vì đây là cơ chế phù hợp cho ứng dụng MVC dạng server-rendered. Hệ thống không cần triển khai JWT hoặc OAuth ở phạm vi hiện tại, do ứng dụng chủ yếu phục vụ phiên đăng nhập trực tiếp trên web. Password được băm bằng BCrypt, một thuật toán phù hợp cho lưu trữ mật khẩu nhờ có salt và chi phí tính toán cao hơn hash thông thường.

Nhìn chung, tổ hợp ASP.NET Core MVC + EF Core + SQLite tạo nên một stack có tính cân bằng cao giữa hiệu năng, độ gọn, tính học thuật và khả năng phát triển tiếp theo. Đây chính là nền tảng để hiện thực hóa khái niệm Localized Data Processing trong bài toán tuyển dụng số.

CHƯƠNG 2. PHÂN TÍCH HỆ THỐNG VÀ MÔ HÌNH HÓA

2.1. Phân tích chức năng

Hệ thống JobFinder được thiết kế như một cổng tuyển dụng tập trung vào ba nhóm chức năng cốt lõi: truy cập và hiển thị việc làm, xác thực người dùng, và điều phối dữ liệu mẫu cho mục đích vận hành hoặc trình diễn.

2.1.1. Nhóm chức năng hiển thị và tra cứu việc làm

Trang chủ của hệ thống đóng vai trò là điểm truy cập chính cho người dùng. Tại đây, danh sách việc làm được nạp từ cơ sở dữ liệu và chuyển tới Razor View thông qua `HomeIndexViewModel`. Cấu trúc này cho phép tách biệt rõ dữ liệu phục vụ giao diện với entity gốc, đồng thời giúp controller chủ động phối hợp nhiều loại dữ liệu trình bày như danh sách kết quả, nhóm công việc nổi bật và từ khóa tìm kiếm hiện hành.

Chức năng tìm kiếm hiện tại được triển khai bằng truy vấn LINQ sử dụng `Contains` trên các trường `Title`, `Description` và `Company`. Đây là chiến lược tìm kiếm theo từ khóa trực tiếp, phù hợp với localized database và tập dữ liệu vừa phải. Truy vấn được thực thi trên `IQueryable`, nghĩa là biểu thức lọc được EF Core chuyển xuống SQLite ở tầng SQL thay vì nạp toàn bộ dữ liệu lên bộ nhớ trước khi lọc. Cách làm này bảo đảm chi phí xử lý hợp lý và giảm bộ nhớ tiêu thụ ở tầng ứng dụng.

Từ góc độ phân tích yêu cầu, bài toán có thể mở rộng sang fuzzy search trong tương lai. Fuzzy search được hiểu là tìm kiếm tương đối, cho phép truy vấn vẫn trả về kết

quả ngay cả khi từ khóa nhập không trùng hoàn toàn với dữ liệu lưu trữ. Trong môi trường localized database, một hướng tiếp cận khả thi là nạp tập con dữ liệu mục tiêu lên bộ nhớ và áp dụng thuật toán đo độ tương đồng chuỗi như Levenshtein distance, Jaro-Winkler hoặc trigram similarity ở tầng ứng dụng. Tuy nhiên, cần nhấn mạnh rằng phiên bản hiện thực của JobFinder vẫn sử dụng tìm kiếm trực tiếp bằng `Contains` để đảm bảo đơn giản, tốc độ và độ ổn định trong bối cảnh đồ án.

2.1.2. Nhóm chức năng xem chi tiết việc làm

Khi người dùng chọn một vị trí tuyển dụng, hệ thống chuyển request sang `JobController` và truy vấn theo khóa chính `Id`. Nếu bản ghi tồn tại, toàn bộ thông tin chi tiết được hiển thị gồm: tiêu đề, công ty, địa điểm, mức lương, ngày đăng, deadline, cấp bậc, kinh nghiệm, loại hình làm việc, mô tả, nhiệm vụ và yêu cầu. Nếu bản ghi không tồn tại, hệ thống trả trạng thái 404 và điều hướng tới trang thông báo phù hợp. Đây là một yêu cầu tưởng như đơn giản nhưng rất quan trọng vì nó phản ánh cách hệ thống xử lý dữ liệu không hợp lệ và duy trì tính nhất quán trong trải nghiệm người dùng.

2.1.3. Cơ chế xác thực claim-based authentication

Phần xác thực được thiết kế theo mô hình cookie authentication kết hợp với claims. Sau khi người dùng gửi form đăng nhập, `AccountController` thực hiện ba bước. Bước một: kiểm tra tính hợp lệ của dữ liệu nhập dựa trên `LoginViewModel` và Data Annotations. Bước hai: tra cứu người dùng trong bảng `Users` theo `Username`, sau đó so sánh mật khẩu bằng `BCrypt.Verify`. Bước ba: nếu xác thực thành công, hệ thống tạo một tập claims bao gồm `NameIdentifier`, `Name`, `Role` và `Username`, đóng gói chúng vào `ClaimsIdentity`, rồi đăng nhập bằng `SignInAsync`.

Ưu điểm của claim-based authentication là thông tin định danh được chuẩn hóa và đính kèm trực tiếp vào `HttpContext.User`. Nhờ đó, view và controller có thể kiểm tra trạng thái đăng nhập hoặc quyền hạn mà không cần truy vấn lại cơ sở dữ liệu trong mọi request. Trong JobFinder, giá trị này được khai thác để hiển thị tên người dùng ở header và để phân nhánh logic reset dữ liệu khi tài khoản có role `Admin`.

Một điểm đáng chú ý là xác thực trong hệ thống không chỉ mang ý nghĩa bảo vệ tài nguyên, mà còn điều phối trạng thái dữ liệu demo. Khi quản trị viên đăng nhập thành công, hệ thống chủ động kích hoạt lại `DataInitializer.Initialize(..., forceReset: true)`. Đây là một quy tắc nghiệp vụ đặc thù, cho thấy xác thực không chỉ dùng để xác nhận danh tính mà còn để kích hoạt hành vi quản trị hệ thống.

2.1.4. Chức năng khởi tạo dữ liệu

`DataInitializer` là một thành phần quan trọng trong hệ thống. Mục tiêu của lớp này là bảo đảm rằng ứng dụng luôn có dữ liệu mẫu nhất quán cho mục đích demo, kiểm thử và vận hành cục bộ. Hàm `Initialize` thực hiện kiểm tra sự tồn tại của dữ liệu, tạo schema nếu database chưa tồn tại, làm sạch dữ liệu cũ nếu cần, và nạp lại danh sách người dùng cùng các bản ghi công việc bằng tiếng Việt.

Từ góc nhìn phân tích chức năng, đây là một dạng "bootstrap workflow" giúp rút ngắn thời gian sẵn sàng vận hành của hệ thống. Người dùng hoặc giảng viên không cần thao tác import dữ liệu bằng tay; chỉ cần khởi động ứng dụng là database sẽ được dựng lên theo đúng trạng thái mẫu.

2.2. Phân tích phi chức năng

2.2.1. Hiệu năng

Trong bối cảnh một portal tuyển dụng cục bộ hoặc quy mô vừa, mục tiêu độ trễ dưới 200ms cho các request đọc đơn giản là hoàn toàn hợp lý. Tốc độ phản hồi phụ thuộc vào nhiều yếu tố: chi phí render Razor, thời gian truy vấn SQLite, số lượng bản ghi, độ phức tạp điều kiện lọc và chi phí truyền tải mạng. Với SQLite chạy cục bộ, chi phí network hop gần như bị triệt tiêu, nhờ đó truy vấn đọc danh sách hoặc chi tiết công việc có thể đạt tốc độ rất tốt nếu số lượng bản ghi còn trong giới hạn hợp lý.

Việc sử dụng `AsNoTracking()` trong các truy vấn đọc giúp giảm chi phí change tracking của EF Core, từ đó cải thiện hiệu năng. Đây là một tối ưu nhỏ nhưng có ý nghĩa trong các trang chủ yếu phục vụ đọc dữ liệu như danh sách việc làm và chi tiết công việc.

2.2.2. Bảo mật

Yêu cầu quan trọng nhất về bảo mật trong phạm vi đồ án là không lưu mật khẩu dưới dạng plain text. JobFinder dùng BCrypt để băm mật khẩu. BCrypt có cơ chế salt và chi phí tính toán tương đối cao, làm tăng độ khó cho các tấn công dò mật khẩu theo từ điển hoặc brute-force nếu dữ liệu bị lộ. So với các hàm hash nhanh như MD5 hoặc SHA-1, BCrypt phù hợp hơn nhiều cho lưu trữ mật khẩu người dùng.

Ở tầng giao diện, form đăng nhập được bảo vệ bằng anti-forgery token. Điều này giúp giảm nguy cơ tấn công CSRF đối với các action POST như login và logout. Bên cạnh đó, cookie authentication cho phép ứng dụng quản lý phiên làm việc theo chuẩn có sẵn của ASP.NET Core, thay vì tự viết cơ chế phiên tùy biến dễ phát sinh lỗi.

2.2.3. Tính tương thích và khả năng triển khai

ASP.NET Core hỗ trợ chạy trên Windows, Linux và môi trường container. Điều này đặc biệt quan trọng khi đồ án được triển khai bằng Docker lên Render. SQLite không yêu cầu dịch vụ nền riêng, vì vậy quá trình triển khai trên container đơn giản hơn đáng kể so với việc phải cấu hình thêm SQL Server hoặc PostgreSQL. Tính tương thích ở đây không chỉ là khả năng chạy trên nhiều hệ điều hành, mà còn là khả năng "đóng gói hoàn chỉnh" của toàn bộ giải pháp.

2.2.4. Tính dễ bảo trì

Mã nguồn được chia theo cấu trúc MVC chuẩn với các thư mục `Controllers`, `Models`, `Data`, `ViewModels`, `Views`, `wwwroot`. Việc phân tách này hỗ trợ tốt cho quá trình đọc hiểu, sửa lỗi và mở rộng. Khi cần thay đổi logic truy vấn, lập trình viên làm việc chủ yếu ở controller hoặc lớp dữ liệu; khi cần thay đổi giao diện, tác động chủ yếu nằm ở view và CSS.

2.3. Thiết kế cơ sở dữ liệu

2.3.1. Bảng Users

Bảng `Users` lưu thông tin xác thực và phân quyền cơ bản. Các trường chính gồm:

- `Id`: khóa chính tự tăng.
- `Username`: định danh đăng nhập, yêu cầu duy nhất ở góc độ nghiệp vụ.

- `PasswordHash`: chuỗi băm mật khẩu.
- `FullName`: họ tên hiển thị.
- `Role`: vai trò người dùng, hiện tại gồm `Admin` và `User`.

Thiết kế này tối giản nhưng đủ cho phạm vi đồ án. Việc tách riêng `PasswordHash` khỏi `Username` và `Role` là cần thiết để duy trì phân lớp dữ liệu bảo mật. Trong tương lai, bảng có thể mở rộng thêm các thuộc tính như email, trạng thái kích hoạt, thời điểm tạo tài khoản hoặc lịch sử đăng nhập.

2.3.2. Bảng Jobs

Bảng `Jobs` là trung tâm của hệ thống và có các trường:

- `Id`
- `Title`
- `Description`
- `Company`
- `Location`
- `Salary`
- `Type`
- `PostedDate`
- `Deadline`
- `Experience`
- `Level`
- `IsRemote`
- `CompanyLogoUrl`
- `Responsibilities`

- `Requirements`

Thiết kế này phản ánh nhu cầu hiển thị của portal tuyển dụng hơn là chuẩn hóa cực đoan dữ liệu. Ví dụ, `Responsibilities` và `Requirements` được lưu dưới dạng chuỗi phân tách ký tự để đơn giản hóa việc seed dữ liệu và render giao diện. Đây không phải là mô hình chuẩn hóa cao nhất, nhưng phù hợp trong một localized portal với mục tiêu tối ưu tốc độ khởi tạo và giảm số lượng bảng quan hệ phụ.

2.3.3. Vai trò của Data Annotations và Fluent API

Trong triển khai hiện tại, hệ thống sử dụng Data Annotations để mô tả ràng buộc dữ liệu như `Required`, `StringLength`, `DataType`, `Display`. Data Annotations đem lại hai lợi ích. Thứ nhất, chúng hỗ trợ validation ở tầng model và được tái sử dụng trong view model cũng như view. Thứ hai, chúng cung cấp metadata để EF Core suy luận schema dữ liệu phù hợp.

Fluent API chưa được sử dụng nhiều trong phiên bản hiện tại vì schema còn tương đối đơn giản. Tuy nhiên, về mặt thiết kế hệ thống, Fluent API vẫn rất quan trọng nếu sau này cần:

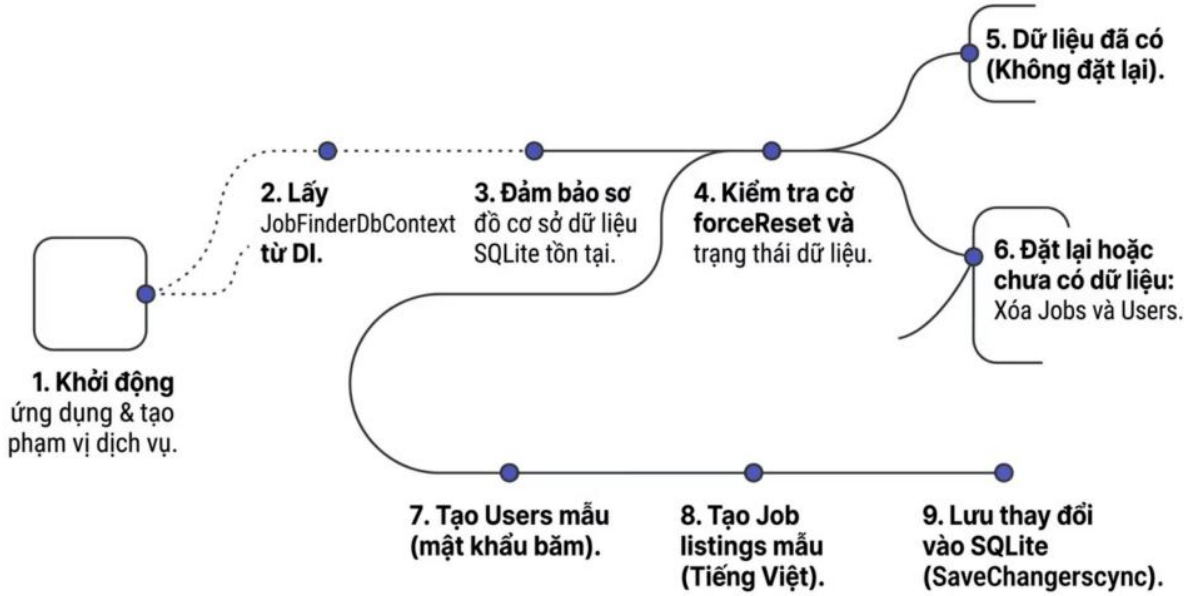
- cấu hình index rõ ràng,
- định nghĩa quan hệ phức tạp,
- đặt ràng buộc unique,
- hoặc tinh chỉnh kiểu dữ liệu cụ thể cho từng cột.

Như vậy, có thể xem Data Annotations là lớp ràng buộc trực tiếp và gần với domain, trong khi Fluent API là công cụ cấu hình mở rộng cho các trường hợp cần kiểm soát chi tiết hơn.

2.4. Mô tả luồng xử lý và sơ đồ logic

2.4.1. Luồng xử lý hàm Initialize Data

Hàm `Initialize` có thể được mô tả theo trình tự sau:



1. Ứng dụng khởi động và tạo scope dịch vụ.
2. Lấy `JobFinderDbContext` từ DI container.
3. Gọi `context.Database.EnsureCreated()` để bảo đảm schema tồn tại.
4. Kiểm tra cờ `forceReset` và trạng thái dữ liệu hiện có.
5. Nếu không cần reset và dữ liệu đã tồn tại, kết thúc hàm.
6. Nếu cần reset hoặc chưa có dữ liệu, xóa toàn bộ dữ liệu trong `Jobs` và `Users`.
7. Tạo tập người dùng mẫu với mật khẩu đã băm.

8. Tạo danh sách việc làm mẫu bằng tiếng Việt.
9. Gọi `SaveChanges()` để ghi dữ liệu xuống SQLite.

Điểm quan trọng của luồng này là tính xác định. Mỗi lần reset, hệ thống quay về cùng một trạng thái dữ liệu chuẩn. Điều này rất hữu ích cho bài toán demo vì mọi người dùng quan sát hệ thống đều làm việc trên cùng một baseline dữ liệu.

2.4.2. Phân tích vấn đề ExecuteDelete và tính toàn vẹn môi trường demo

Về mặt lý thuyết, trong EF Core 7 trở lên, `ExecuteDelete()` là một lựa chọn tối ưu cho kịch bản làm sạch dữ liệu hàng loạt vì thao tác được đẩy trực tiếp xuống SQL mà không cần nạp entity lên memory. Trong môi trường demo, phương pháp này có ưu điểm rõ: nhanh, gọn, giảm overhead change tracking và phù hợp với quy trình reset toàn bộ dữ liệu trước khi seed lại. Đây chính là lý do nhiều tài liệu hoặc thiết kế mô tả bước "clean then seed" thường khuyến nghị `ExecuteDelete()` hoặc cơ chế truncate tương đương.

Tuy nhiên, cần phân biệt giữa phân tích thiết kế và hiện thực thực tế. Phiên bản JobFinder trong mã nguồn hiện tại dùng EF Core 6 nên chưa có `ExecuteDelete()`. Vì vậy, giải pháp đang dùng là `RemoveRange(context.Jobs)` và `RemoveRange(context.Users)` rồi `SaveChanges()`. Cách tiếp cận này vẫn đảm bảo đúng chức năng ở quy mô đồ án, dù kém tối ưu hơn về mặt chi phí thực thi.

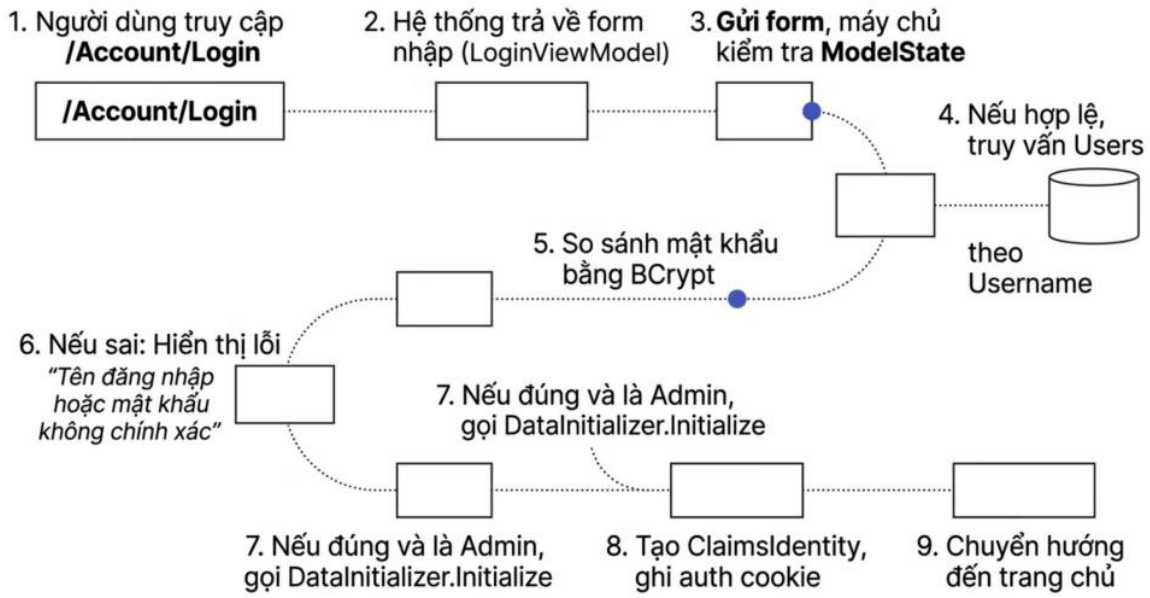
Từ góc nhìn toàn vẹn môi trường demo, mục tiêu quan trọng không chỉ là xóa nhanh, mà là bảo đảm rằng không còn dữ liệu cũ gây nhiễu trước khi sinh lại dữ liệu mới. Nếu một phần dữ liệu cũ còn sót lại, hệ thống có thể gặp các vấn đề như:

- danh sách việc làm bị trùng lặp,
- tài khoản quản trị tồn tại ở nhiều trạng thái,
- thống kê hoặc thứ tự hiển thị không còn nhất quán,
- các kiểm thử giao diện dựa trên số lượng bản ghi bị sai lệch.

Do đó, chiến lược "delete all before seed" là hợp lý cho hệ thống phục vụ demo, miễn là được thực hiện có chủ đích và chỉ áp dụng trong môi trường kiểm soát.

2.4.3. Luồng backend đăng nhập

Luồng đăng nhập có thể mô tả như sau:



Thiết kế này cho thấy backend không chỉ xử lý logic nghiệp vụ mà còn phối hợp chặt với mục tiêu vận hành demo của hệ thống.

CHƯƠNG 3. TRIỂN KHAI KỸ THUẬT VÀ THỰC NGHIỆM

3.1. Kiến trúc frontend

Frontend của JobFinder sử dụng Razor View kết hợp HTML, CSS và JavaScript thuần. Cách tiếp cận này phù hợp với mô hình MVC server-rendered, giúp giảm độ phức tạp so với việc tách frontend SPA riêng trong khi vẫn đủ linh hoạt để tạo giao diện hiện đại.

3.1.1. Kỹ thuật Glassmorphism trên trang Login

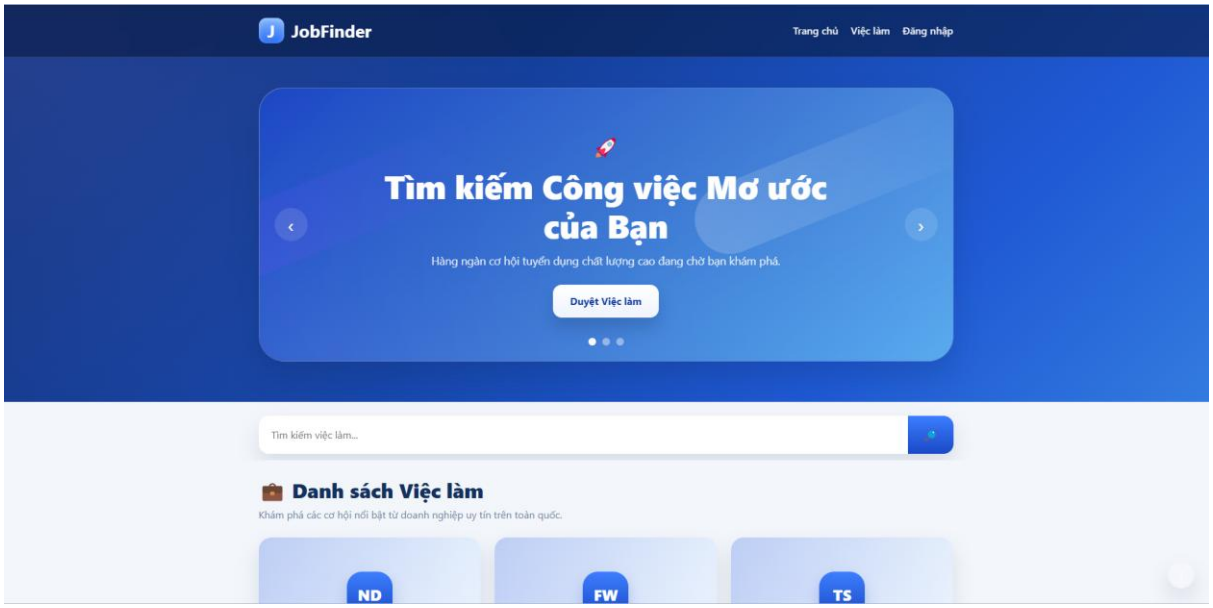
Trang đăng nhập là một trong những điểm thể hiện rõ tính thẩm mỹ của hệ thống. Giao diện được thiết kế theo ngôn ngữ hình ảnh xanh lam nhiều lớp, gợi cảm giác chuyên nghiệp và tin cậy, đồng thời áp dụng hiệu ứng Glassmorphism cho khối form trung tâm. Về kỹ thuật CSS, hiệu ứng này được tạo thông qua ba yếu tố:

- nền card bán trong suốt với `rgba`,
- đường viền sáng mờ tạo cảm giác tách lớp,
- `backdrop-filter: blur(...)` để làm mờ phần nền phía sau card.

Khi kết hợp với nền gradient nhiều lớp, card đăng nhập tạo ra hiệu ứng thị giác giống kính mờ. Điều này không chỉ có giá trị trang trí mà còn giúp người dùng tập trung vào form mà không cần cắt bỏ hoàn toàn chiều sâu của nền. Trong bối cảnh portal tuyển dụng, một giao diện đủ hiện đại sẽ góp phần củng cố cảm nhận tích cực về thương hiệu tuyển dụng.

Ở góc độ triển khai, Glassmorphism yêu cầu chú ý đến khả năng tương thích trình duyệt. Thuộc tính `backdrop-filter` hoạt động tốt trên phần lớn trình duyệt hiện đại, nhưng vẫn cần fallback hợp lý bằng màu nền bán trong suốt nếu trình duyệt không hỗ trợ đầy đủ. Phiên bản hiện tại của JobFinder đạt được sự cân bằng chấp nhận được giữa tính hiện đại và khả năng hiển thị.

3.1.2. Slider động bằng JavaScript kết hợp Razor View



Trang chủ sử dụng một hero slider để mô tả thông điệp tuyển dụng. Dữ liệu slide được định nghĩa trực tiếp trong Razor View dưới dạng một mảng đối tượng, sau đó vòng lặp Razor sinh ra từng slide tương ứng. Cách làm này có hai ưu điểm. Thứ nhất, nội dung của slide vẫn được quản lý trong lớp view server-side, dễ đồng bộ với ngôn ngữ và logic hiển thị. Thứ hai, phần chuyển trạng thái giữa các slide được JavaScript xử lý ở phía client, tạo cảm giác động mà không phải tải lại trang.

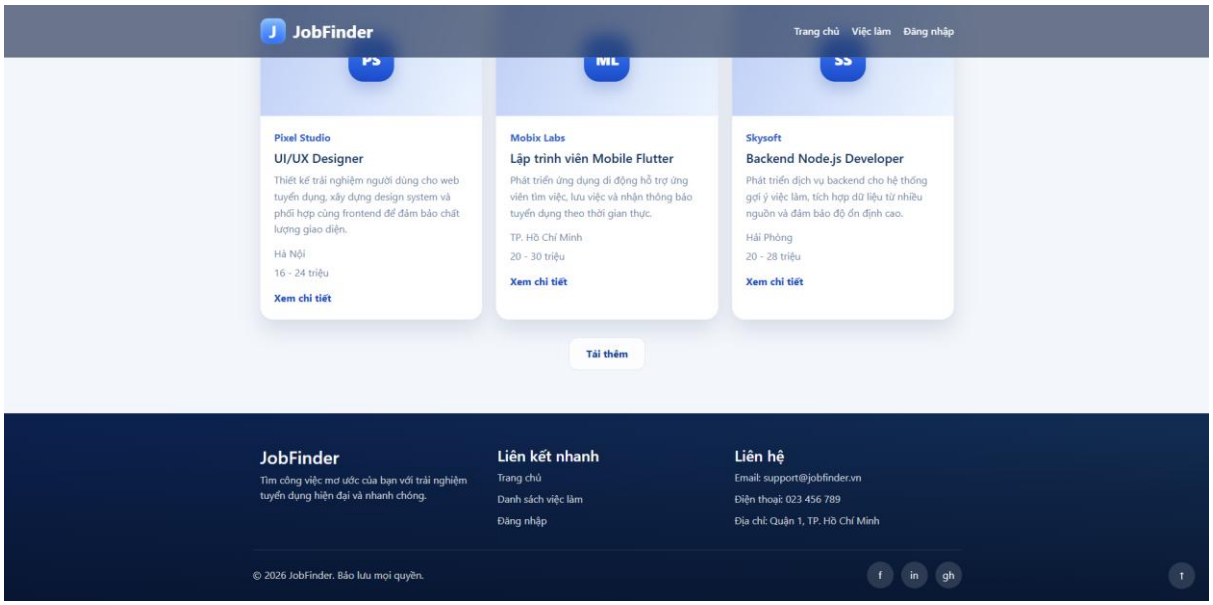
Trong `site.js`, slider được điều khiển bằng cách:

- gom danh sách phần tử slide,
- lưu chỉ số slide đang hoạt động,

- chuyển class `is-active` giữa các slide và dot tương ứng,
- bắt sự kiện click cho nút next/prev,
- tự động xoay slide theo chu kỳ `setInterval`.

Mô hình này đơn giản nhưng hiệu quả. Nó tránh phụ thuộc vào thư viện slider nặng, giảm chi phí tài nguyên và kiểm soát tốt hơn cấu trúc DOM. Với một hệ thống tuyển dụng đặt nặng tốc độ tải trang, việc tránh thêm nhiều thư viện frontend là một lựa chọn hợp lý.

3.1.3. Kỹ thuật tải thêm danh sách việc làm



Chức năng "Tải thêm" được hiện thực ở phía client bằng cách ẩn/hiện dần các phần tử card việc làm. Ban đầu, một số card đầu được hiển thị; khi người dùng nhấn nút, JavaScript tăng số lượng card được gắn class `is-visible`. Đây là một chiến lược tiết kiệm hơn so với gọi API phân trang trong phạm vi demo vì toàn bộ danh sách đã có sẵn từ server, còn nhu cầu chủ yếu là cải thiện cảm nhận duyệt nội dung.

3.2. Logic backend

3.2.1. Dependency Injection với DbContext

Ứng dụng không triển khai Generic Repository pattern mà sử dụng trực tiếp `JobFinderDbContext` thông qua Dependency Injection. Đây là một quyết định kỹ thuật có chủ đích. Trong các ứng dụng nhỏ và vừa dùng EF Core, việc thêm một lớp repository tổng quát đôi khi không tạo thêm giá trị mà còn làm tăng số lượng abstraction không cần thiết. EF Core bản thân đã cung cấp một abstraction khá đầy đủ thông qua `DbContext` và `DbSet`.

Trong `Program.cs`, `JobFinderDbContext` được đăng ký bằng:

```
builder.Services.AddDbContext<JobFinderDbContext>(options =>
options.UseSqlite(builder.Configuration.GetConnectionString("Defaul
tConnection")));
```

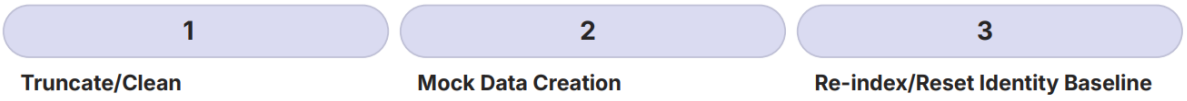
Sau đó, controller nhận `JobFinderDbContext` qua constructor injection. Cách làm này đem lại các lợi ích:

- tận dụng vòng đời scoped mặc định phù hợp cho mỗi request web,
- tránh khởi tạo context thủ công trong controller,
- dễ thay đổi provider database khi cần nâng cấp,
- thuận tiện cho kiểm thử nếu sau này cần mock hoặc thay bằng context thử nghiệm.

`HomeController`, `JobController` và `AccountController` đều sử dụng `DbContext` theo hướng "thin controller" tương đối rõ ràng: nhận request, truy vấn dữ liệu, dựng view model hoặc claims, rồi trả view/redirect. Đây là cấu trúc phù hợp cho đồ án vì dễ theo dõi và không làm lớp điều phối bị phình quá mức.

3.2.2. Phân tích sâu hàm InitData: Truncate - Mock Data - Re-index

Ở mức logic khái niệm, `InitData` có thể được mô tả bởi chuỗi ba bước:



10. Truncate/Clean
11. Mock Data Creation
12. Re-index/Reset Identity Baseline

Trong hiện thực với SQLite và EF Core 6, bước một được mô phỏng bằng `RemoveRange` thay vì `TRUNCATE TABLE` thực thụ. Dù vậy, ý nghĩa nghiệp vụ không thay đổi: loại bỏ hoàn toàn dữ liệu cũ. Bước hai tạo lại danh sách bản ghi theo bộ dữ liệu mẫu tiếng Việt đã được chuẩn hóa về nội dung. Bước ba không được cài đặt dưới dạng lệnh `REINDEX` SQL riêng trong mã nguồn, nhưng về mặt vận hành, việc tạo mới database và seed lại giúp trạng thái bản ghi quay về baseline nhất quán phục vụ demo. Trong môi trường SQLite, việc "re-index" ở đây nên hiểu theo nghĩa khôi phục trạng thái logic của tập dữ liệu hơn là thao tác tái cấu trúc index vật lý một cách bắt buộc.

Một điểm thiết kế đáng chú ý là luồng reset được gắn với tài khoản `Admin`. Điều này giúp người dùng thường không thể vô tình làm thay đổi trạng thái dataset chuẩn. Cơ chế này biến `InitData` thành một thao tác quản trị được kiểm soát bởi xác thực, thay vì một script vận hành tách biệt bên ngoài ứng dụng.

3.2.3. Luồng backend đăng nhập

Luồng đăng nhập có thể mô tả như sau:

13. Người dùng truy cập `/Account/Login`.
14. Hệ thống trả form nhập liệu dựa trên `LoginViewModel`.
15. Khi submit, server kiểm tra `ModelState`.
16. Nếu dữ liệu hợp lệ, truy vấn bảng `Users` theo `Username`.
17. So sánh mật khẩu nhập với `PasswordHash` bằng BCrypt.
18. Nếu sai, đưa lỗi "Tên đăng nhập hoặc mật khẩu không chính xác".
19. Nếu đúng và role là `Admin`, gọi `DataInitializer.Initialize(..., forceReset: true)`.
20. Tạo `ClaimsIdentity`, ghi auth cookie.
21. Redirect về trang chủ.

Thiết kế này cho thấy backend không chỉ xử lý logic nghiệp vụ mà còn phối hợp chặt với mục tiêu vận hành demo của hệ thống.

3.3. Đánh giá thực nghiệm

3.3.1. Phương pháp thử nghiệm

Để đánh giá tính khả thi của localized database, có thể xây dựng kịch bản thử nghiệm với khoảng 10.000 bản ghi `Jobs`. Các phép đo nên tập trung vào:

- thời gian trả về danh sách công việc theo truy vấn không điều kiện,
- thời gian truy vấn theo từ khóa,
- thời gian lấy chi tiết một bản ghi theo khóa chính,
- chi phí khởi tạo dữ liệu ban đầu,
- mức sử dụng bộ nhớ ở tầng ứng dụng.

Trong kịch bản này, SQLite cần được đặt cùng môi trường thực thi với ứng dụng để phản ánh đúng lợi thế localized data. Chỉ số nên được đo nhiều lần, loại bỏ sai số của lần chạy đầu do warm-up runtime.

3.3.2. Kỳ vọng hiệu năng của SQLite ở mức 10.000 bản ghi

Với bài toán chỉ đọc danh sách và chi tiết theo khóa chính, 10.000 bản ghi là quy mô hoàn toàn trong khả năng xử lý tốt của SQLite. Nếu dữ liệu được mô hình hóa hợp lý, truy vấn chi tiết theo `Id` gần như có độ trễ rất thấp vì dựa trên khóa chính. Truy vấn tìm kiếm theo `Contains` sẽ phụ thuộc vào độ dài dữ liệu văn bản và khả năng tối ưu của SQLite, nhưng ở mức 10.000 bản ghi vẫn thường cho thời gian chấp nhận được trong môi trường cục bộ.

So với các hệ quản trị như SQL Server hoặc PostgreSQL, SQLite không mạnh bằng khi số lượng truy cập đồng thời lớn hoặc khi cần tối ưu khóa, transaction phức tạp, replication hay phân tán dữ liệu. Tuy nhiên, ở bài toán localized portal với tải vừa phải, lợi thế của SQLite nằm ở:

- không có chi phí round-trip tới database server riêng,
- không cần lớp hạ tầng quản trị độc lập,

- thời gian khởi tạo môi trường cực thấp,
- đơn giản hóa sao lưu và đóng gói.

Do đó, nếu tiêu chí đánh giá bao gồm cả tổng chi phí sở hữu và tốc độ triển khai, SQLite là một lựa chọn rất cạnh tranh ở giai đoạn đầu.

3.3.3. So sánh định tính với các giải pháp khác

So với SQL Server:

- SQL Server mạnh hơn về transaction, bảo mật doanh nghiệp, concurrency và quản trị.
- SQLite nhẹ hơn nhiều, phù hợp cho demo, PoC, private portal quy mô vừa.

So với Oracle:

- Oracle vượt trội ở bài toán enterprise mission-critical.
- Chi phí vận hành và độ phức tạp quá lớn nếu áp dụng cho một portal tuyển dụng localized.

So với file JSON hoặc in-memory store:

- SQLite có ưu thế về ngôn ngữ truy vấn, tính bền vững dữ liệu, ràng buộc schema và khả năng mở rộng tốt hơn.

Như vậy, trong phạm vi đồ án, SQLite là điểm cân bằng giữa tính kỹ thuật và tính thực dụng.

3.4. Giao diện người dùng và định hướng thiết kế

Thiết kế giao diện của JobFinder sử dụng tông xanh chủ đạo, được mô tả trong báo cáo là Professional Blue. Đây là một quyết định có cơ sở từ góc nhìn trải nghiệm người dùng. Trong các sản phẩm tuyển dụng, màu xanh thường gắn với cảm nhận tin cậy, ổn định, chuyên nghiệp và ít gây áp lực thị giác hơn so với các gam màu quá nóng. Màu nền gradient nhiều lớp kết hợp với ánh sáng mềm tạo cảm giác hiện đại nhưng vẫn nghiêm túc, phù hợp với lĩnh vực nhân sự.

Layout tổng thể được tổ chức theo trục dọc rõ ràng:

- header chứa logo và điều hướng,
- hero/banner làm khối thu hút đầu tiên,
- thanh tìm kiếm ở vị trí ngay dưới banner để tối ưu khả năng hành động,
- vùng danh sách việc làm dạng card,
- footer chứa thông tin liên hệ và liên kết nhanh.

Cách bố trí này phản ánh nguyên tắc thị giác "from message to action": trước hết giới thiệu giá trị của hệ thống, sau đó đưa người dùng vào thao tác tìm kiếm, và cuối cùng cung cấp kết quả dưới dạng card có khả năng quét nhanh.

Ở trang chi tiết công việc, bố cục hai cột được sử dụng để phân tách nội dung chính với thông tin phụ trợ. Cột trái dành cho mô tả, nhiệm vụ, yêu cầu và hành động ứng

tuyển; cột phải dành cho thông tin công ty, điểm nhấn vị trí và khu vực chia sẻ. Bố cục này tương thích với thói quen đọc nội dung dài trên web: người dùng quét tổng quan ở khối chính trước, sau đó xem metadata và hành động ở sidebar.

3.4.1. Tính nhất quán giữa giao diện và backend

Một điểm đáng chú ý là giao diện của JobFinder không được thiết kế tách rời backend. Các quyết định về schema dữ liệu đã ảnh hưởng trực tiếp tới cách hiển thị. Ví dụ:

- `Responsibilities` và `Requirements` được lưu dạng chuỗi phân tách, sau đó tách thành danh sách trong Razor View.
- `CompanyLogoUrl` hiện được sử dụng như một ký hiệu chữ viết tắt thay vì URL ảnh tuyệt đối, giúp seed dữ liệu nhanh và giao diện vẫn giữ được nhận diện công ty.
- `Role` của người dùng ảnh hưởng trực tiếp tới header và các hành vi sau đăng nhập.

Điều này cho thấy đồ án không chỉ là ghép một giao diện vào một backend có sẵn, mà là quá trình đồng thiết kế giữa mô hình dữ liệu, logic xử lý và trải nghiệm hiển thị.

3.4.2. Khả năng responsive và triển khai thực tế

CSS của hệ thống đã bổ sung các breakpoint ở các mức phổ biến nhằm bảo đảm giao diện co giãn hợp lý trên thiết bị di động và màn hình nhỏ. Việc một portal tuyển dụng hiển thị tốt trên mobile là yêu cầu thực tế vì phần lớn ứng viên hiện nay tra cứu việc làm qua điện thoại. Trong đồ án, responsive layout được xử lý theo hướng giảm số cột của grid, thu gọn padding và điều chỉnh kích thước vùng tương tác, thay vì thiết kế

hai hệ giao diện hoàn toàn khác nhau. Đây là giải pháp hợp lý cho phạm vi triển khai hiện tại.

KẾT LUẬN

Đồ án "Hệ thống điều phối nhân sự JobFinder trên nền tảng ASP.NET Core & Localized Database" đã đạt được các mục tiêu cốt lõi đề ra. Thứ nhất, hệ thống đã chứng minh tính khả thi của mô hình localized data processing trong bài toán portal tuyển dụng. Bằng việc sử dụng SQLite kết hợp với Entity Framework Core, đồ án đã hiện thực được một ứng dụng vận hành với cấu hình nhẹ, không phụ thuộc vào máy chủ cơ sở dữ liệu tập trung riêng biệt, nhưng vẫn cung cấp đầy đủ các chức năng quan trọng như xác thực người dùng, tìm kiếm việc làm, xem chi tiết vị trí và khởi tạo dữ liệu nhất quán.

Thứ hai, đồ án đã xây dựng được một kiến trúc phần mềm rõ ràng dựa trên ASP.NET Core MVC. Việc phân tách controller, model, view, view model và lớp dữ liệu giúp hệ thống dễ hiểu, dễ bảo trì và có khả năng mở rộng. Các kỹ thuật như cookie authentication theo claims, băm mật khẩu bằng BCrypt, Data Annotations cho validation, và Dependency Injection với `DbContext` đã được áp dụng đúng hướng, bảo đảm hệ thống không chỉ chạy được mà còn phản ánh các nguyên tắc nền tảng của phát triển phần mềm hiện đại.

Thứ ba, đồ án đã kết hợp được giữa yếu tố kỹ thuật backend và trải nghiệm người dùng frontend. Giao diện theo tông Professional Blue, trang đăng nhập Glassmorphism, slider ở trang chủ, bố cục chi tiết việc làm hai cột và cơ chế tải thêm danh sách là những thành phần thể hiện rằng một hệ thống gọn nhẹ vẫn có thể mang lại trải nghiệm trực quan và có tính ứng dụng thực tiễn.

Thứ tư, hệ thống đã chứng minh giá trị triển khai trong môi trường container hóa. Với cấu hình Docker và Render, JobFinder có thể được đóng gói và phân phối nhanh, trong khi SQLite vẫn hoạt động hiệu quả nhờ cơ chế mount persistent disk. Điều này cho thấy localized database không đối lập với triển khai hiện đại; ngược lại, nếu được thiết kế đúng, nó có thể trở thành một chiến lược rất thực dụng cho các ứng dụng quy mô vừa và nhỏ.

Tuy nhiên, báo cáo cũng chỉ ra rõ giới hạn của hướng tiếp cận này. SQLite không phải lựa chọn tối ưu nếu hệ thống phát triển đến mức có số lượng người dùng đồng thời lớn, nhu cầu transaction phức tạp, yêu cầu phân tán dữ liệu hoặc nhu cầu phân tích nâng cao trên tập dữ liệu khổng lồ. Bên cạnh đó, cơ chế tìm kiếm hiện tại mới dừng ở mức từ khóa trực tiếp và chưa triển khai fuzzy search hoặc full-text search chuyên sâu.

Hướng phát triển

Trong tương lai, hệ thống có thể phát triển theo các hướng sau:

- Nâng cấp runtime lên .NET 8 để tận dụng hiệu năng mới và vòng đời hỗ trợ dài hơn.
- Bổ sung phân hệ quản trị việc làm, quản lý hồ sơ ứng viên và dashboard thống kê.
- Tách lớp dịch vụ nghiệp vụ rõ hơn nếu hệ thống bắt đầu mở rộng nhiều module.
- Triển khai fuzzy search hoặc full-text search cho trải nghiệm tìm kiếm tốt hơn.
- Áp dụng Fluent API để cấu hình index, unique constraint và các quan hệ dữ liệu phức tạp hơn.

- Chuyển dần từ monolithic MVC sang kiến trúc dịch vụ hoặc microservices khi quy mô nghiệp vụ tăng.
- Thay thế localized SQLite bằng cloud-native database như Azure SQL, PostgreSQL hoặc dịch vụ phân tán khác khi dung lượng dữ liệu, số lượng truy cập và yêu cầu đồng thời vượt ngưỡng phù hợp của file-based database.

Tóm lại, JobFinder không chỉ là một ứng dụng tuyển dụng mẫu, mà còn là một minh chứng kỹ thuật cho thấy với chiến lược lựa chọn công nghệ đúng đắn, một hệ thống phần mềm có thể đạt được sự cân bằng giữa hiệu năng, chi phí, độ gọn và khả năng mở rộng. Đây là giá trị cốt lõi mà đồ án hướng tới.

TÀI LIỆU THAM KHẢO

22. Microsoft. ASP.NET Core documentation.
23. Microsoft. Entity Framework Core documentation.
24. SQLite Documentation. SQLite Home Page and Technical Notes.
25. OWASP Foundation. Password Storage Cheat Sheet.
26. Gamma, E., Helm, R., Johnson, R., Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software.
27. Pressman, R. S. Software Engineering: A Practitioner's Approach.
28. Sommerville, I. Software Engineering.
29. Các tài liệu học thuật và kỹ thuật liên quan đến E-Recruitment, web application architecture và database deployment patterns.